

4.7.4 Quadratic Discriminant Analysis

Bây giờ ta sẽ fit một QDA model cho dữ liệu `Smarket`. QDA được triển khai bằng `QuadraticDiscriminantAnalysis()` trong package `sklearn`, mà ta viết tắt là `QDA()`. Syntax rất giống `LDA()`.

*Quadratic
Discriminant
Analysis()*

```
In [33]: qda = QDA(store_covariance=True)
         qda.fit(X_train, L_train)
```

```
Out[33]: QuadraticDiscriminantAnalysis(store_covariance=True)
```

Function `QDA()` một lần nữa sẽ tính `means_` và `priors_`.

```
In [34]: qda.means_, qda.priors_
```

```
Out[34]: (array([[ 0.04279022,  0.03389409],
                [-0.03954635, -0.03132544]]),
         array([0.49198397, 0.50801603]))
```

Classifier `QDA()` sẽ estimate một covariance cho mỗi class. Dưới đây là estimated covariance của class thứ nhất:

```
In [35]: qda.covariance_[0]
```

```
Out[35]: array([[ 1.50662277, -0.03924806],
                [-0.03924806,  1.53559498]])
```

Output chứa các group mean. Tuy nhiên, nó không chứa các coefficient của linear discriminant, vì QDA classifier sử dụng một hàm quadratic của các predictor thay vì một hàm linear. Function `predict()` hoạt động hoàn toàn giống như với LDA.

```
In [36]: qda_pred = qda.predict(X_test)
         confusion_table(qda_pred, L_test)
```

```
Out[36]:      Truth  Down  Up
Predicted
   Down      30   20
   Up       81  121
```

Điều thú vị là prediction của QDA chính xác gần 60% số lần, mặc dù dữ liệu năm 2005 không được dùng để fit model.

```
In [37]: np.mean(qda_pred == L_test)
```

```
Out[37]: 0.599
```

Mức accuracy này khá ấn tượng đối với stock market data, vốn nổi tiếng là rất khó model chính xác. Kết quả này gợi ý rằng dạng quadratic mà QDA giả định có thể nắm bắt relationship thực sự chính xác hơn các dạng linear mà LDA và logistic regression giả định. Tuy nhiên, ta khuyến nghị đánh giá performance của method này trên một test set lớn hơn trước khi đặt cược rằng approach này sẽ liên tục đánh bại thị trường!

4.7.5 Naive Bayes

Tiếp theo, ta fit một naive Bayes model cho dữ liệu **Smarket**. Syntax tương tự `LDA()` và `QDA()`. Theo mặc định, implementation `GaussianNB()` của naive Bayes classifier model mỗi quantitative feature bằng một Gaussian distribution. Tuy nhiên, cũng có thể dùng một kernel density method để estimate các distribution.

GaussianNB()

In [38]:

```
NB = GaussianNB()
NB.fit(X_train, L_train)
```

Out[38]:

GaussianNB()

Các class được lưu trong `classes_`.

In [39]:

```
NB.classes_
```

Out[39]:

```
array(['Down', 'Up'], dtype='<U4')
```

Class prior probability được lưu trong attribute `class_prior_`.

In [40]:

```
NB.class_prior_
```

Out[40]:

```
array([0.49, 0.51])
```

Các parameter của feature có thể được tìm thấy trong các attribute `theta_` và `var_`. Số row bằng số class, còn số column bằng số feature. Ta thấy bên dưới rằng mean của feature **Lag1** trong class **Down** là 0.043.

In [41]:

```
NB.theta_
```

Out[41]:

```
array([[ 0.043,  0.034],
       [-0.040, -0.031]])
```

Variance của nó là 1.503.

In [42]:

```
NB.var_
```

Out[42]:

```
array([[1.503, 1.532],
       [1.514, 1.487]])
```

Làm sao ta biết tên của các attribute này? Ta dùng `NB?` (hoặc `?NB`). Ta có thể dễ dàng verify phép tính mean:

In [43]:

```
X_train[L_train == 'Down'].mean()
```

Out[43]:

```
Lag1    0.042790
Lag2    0.033894
dtype: float64
```

Tương tự đối với variance:

```
In [44]: X_train[L_train == 'Down'].var(ddof=0)
```

```
Out[44]: Lag1      1.503554
Lag2      1.532467
dtype: float64
```

Function `GaussianNB()` tính variance bằng công thức $1/n$.⁶ Vì `NB()` là một classifier trong thư viện `sklearn`, việc tạo prediction dùng cùng syntax như với `LDA()` và `QDA()` ở trên.

```
In [45]: nb_labels = NB.predict(X_test)
confusion_table(nb_labels, L_test)
```

```
Out[45]:      Truth  Down  Up
Predicted
Down      29   20
Up       82  121
```

Naive Bayes hoạt động tốt trên các dữ liệu này, với prediction chính xác hơn 59% số lần. Kết quả này hơi kém QDA nhưng tốt hơn LDA nhiều.

Tương tự LDA, method `predict_proba()` estimate probability rằng mỗi observation thuộc về một class cụ thể.

```
In [46]: NB.predict_proba(X_test)[:5]
```

```
Out[46]: array([[0.4873, 0.5127],
 [0.4762, 0.5238],
 [0.4653, 0.5347],
 [0.4748, 0.5252],
 [0.4902, 0.5098]])
```

4.7.6 *K-Nearest Neighbors*

Bây giờ ta sẽ thực hiện KNN bằng function `KNeighborsClassifier()`. Function này hoạt động tương tự các model-fitting function khác mà ta đã gặp cho đến đây.

Giống như với LDA và QDA, ta fit classifier bằng method `fit`. Prediction mới được tạo bằng method `predict` của object do `fit()` trả về.

*KNeighbors
Classifier()*

```
In [47]: knn1 = KNeighborsClassifier(n_neighbors=1)
knn1.fit(X_train, L_train)
knn1_pred = knn1.predict(X_test)
confusion_table(knn1_pred, L_test)
```

```
Out[47]:      Truth  Down  Up
Predicted
Down      43   58
Up       68   83
```

Kết quả với $K = 1$ không tốt lắm, vì chỉ 50% observation được dự đoán

⁶Có hai công thức để tính sample variance của n observation $x_1, \dots, x_n$: $\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$ và $\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$, trong đó $\bar{x}$ là sample mean. Trong hầu hết trường hợp, sự khác biệt này không quan trọng.

đúng. Tất nhiên, có thể $K = 1$ tạo ra một fit quá flexible đối với dữ liệu.

```
In [48]: (83+43)/252, np.mean(knn1_pred == L_test)
```

```
Out[48]: (0.5, 0.5)
```

Ta lặp lại analysis bên dưới với $K = 3$.

```
In [49]: knn3 = KNeighborsClassifier(n_neighbors=3)
knn3_pred = knn3.fit(X_train, L_train).predict(X_test)
np.mean(knn3_pred == L_test)
```

```
Out[49]: 0.532
```

Kết quả đã cải thiện đôi chút. Tuy nhiên, tăng K thêm nữa không đem lại cải thiện nào khác. Có vẻ rằng với dữ liệu này và train/test split này, QDA cho kết quả tốt nhất trong số các method mà ta đã khảo sát cho đến đây.

KNN không hoạt động tốt trên dữ liệu **Smarket**, nhưng nó thường có thể cho kết quả ấn tượng. Ví dụ, ta sẽ áp dụng KNN approach cho dataset **Caravan**, một phần của thư viện ISLP. Dataset này gồm 85 predictor đo các đặc trưng nhân khẩu học của 5,822 cá nhân. Response variable là **Purchase**, cho biết một cá nhân có mua một hợp đồng bảo hiểm caravan hay không. Trong dataset này, chỉ 6% số người mua bảo hiểm caravan.

```
In [50]: Caravan = load_data('Caravan')
Purchase = Caravan.Purchase
Purchase.value_counts()
```

```
Out[50]: No      5474
Yes       348
Name: Purchase, dtype: int64
```

Method `value_counts()` nhận một `pd.Series` hoặc `pd.DataFrame` và trả về một `pd.Series` với các count tương ứng cho từng unique element. Trong trường hợp này, **Purchase** chỉ có các giá trị **Yes** và **No**, và trả về số lượng của từng giá trị.

```
In [51]: 348 / 5822
```

```
Out[51]: 0.0598
```

Các feature của ta sẽ gồm mọi column ngoại trừ **Purchase**.

```
In [52]: feature_df = Caravan.drop(columns=['Purchase'])
```

Vì KNN classifier dự đoán class của một test observation bằng cách xác định các observation gần nó nhất, scale của các variable rất quan trọng. Bất kỳ variable nào có scale lớn sẽ có ảnh hưởng lớn hơn nhiều đến distance giữa các observation, và do đó đến KNN classifier, so với variable có scale nhỏ. Chẳng hạn, hãy tưởng tượng một dataset có hai variable là lương và tuổi (được đo lần lượt bằng đô la và năm). Đối với KNN, chênh lệch 1,000

USD về lương là cực lớn so với chênh lệch 50 năm về tuổi. Do đó lương sẽ chi phối kết quả KNN classification, còn tuổi gần như không có ảnh hưởng. Điều này trái với trực giác rằng chênh lệch 1,000 USD về lương khá nhỏ so với chênh lệch 50 năm về tuổi. Hơn nữa, tầm quan trọng của scale đối với KNN classifier dẫn đến một vấn đề khác: nếu ta đo lương bằng yên Nhật, hoặc đo tuổi bằng phút, thì kết quả classification sẽ rất khác so với khi hai variable này được đo bằng đô la và năm.

Một cách tốt để xử lý vấn đề này là standardize dữ liệu sao cho mọi variable đều có mean bằng 0 và standard deviation bằng 1. Khi đó mọi variable sẽ ở scale có thể so sánh được. Việc này được thực hiện bằng transformation `StandardScaler()`.

standardize

In [53]:

```
scaler = StandardScaler(with_mean=True,
                        with_std=True,
                        copy=True)
```

*Standard
Scaler()*

Argument `with_mean` cho biết có trừ mean hay không, còn `with_std` cho biết có scale các column để có standard deviation bằng 1 hay không. Cuối cùng, argument `copy=True` cho biết ta luôn copy dữ liệu thay vì cố thực hiện calculation in place khi có thể.

Transformation này có thể được fit rồi áp dụng cho dữ liệu tùy ý. Ở dòng đầu bên dưới, các parameter cho scaling được tính và lưu trong `scaler`; dòng thứ hai thực sự dùng standardized feature set.

In [54]:

```
scaler.fit(feature_df)
X_std = scaler.transform(feature_df)
```

Bây giờ mọi column của `feature_std` bên dưới đều có standard deviation bằng 1 và mean bằng 0.

In [55]:

```
feature_std = pd.DataFrame(
    X_std,
    columns=feature_df.columns);
feature_std.std()
```

Out[55]:

```
MOSTYPE      1.000086
MAANTHUI     1.000086
MGEMOMV      1.000086
MGEMLEEF     1.000086
MOSHOOFD     1.000086
...
AZEILPL      1.000086
APLEZIER     1.000086
AFIETS       1.000086
AINBOED      1.000086
ABYSTAND     1.000086
Length: 85, dtype: float64
```

Lưu ý rằng standard deviation ở đây không hoàn toàn bằng 1; nguyên nhân một lần nữa là do một số procedure dùng convention $1/n$ cho variance (trong trường hợp này là `scaler()`), trong khi những procedure khác dùng $1/(n-1)$ (method `std()`). Xem footnote ở trang 183. Trong trường hợp này điều đó không quan trọng, miễn là mọi variable đều ở cùng scale.

.std()

Dùng function `train_test_split()`, bây giờ ta chia các observation thành một test set chứa 1000 observation và một training set chứa các

observation còn lại. Argument `random_state=0` bảo đảm rằng ta nhận được cùng một split mỗi lần chạy lại code.

train_test_split()

In [56]:

```
(X_train,
 X_test,
 y_train,
 y_test) = train_test_split(feature_std,
                             Purchase,
                             test_size=1000,
                             random_state=0)
```

?`train_test_split` cho thấy các non-keyword argument có thể là list, array, pandas dataframe, v.v., tất cả đều có cùng length (`shape[0]`) và vì thế đều indexable. Trong trường hợp này chúng là dataframe `feature_std` và response variable `Purchase`. Ta fit một KNN model trên training data với $K = 1$, rồi đánh giá performance của nó trên test data.

indexable

In [57]:

```
knn1 = KNeighborsClassifier(n_neighbors=1)
knn1_pred = knn1.fit(X_train, y_train).predict(X_test)
np.mean(y_test != knn1_pred), np.mean(y_test != "No")
```

Out[57]:

(0.111, 0.067)

KNN error rate trên 1,000 test observation xấp xỉ 11%. Thoạt nhìn, kết quả này có vẻ khá tốt. Tuy nhiên, vì chỉ hơn 6% khách hàng mua bảo hiểm, ta có thể giảm error rate xuống gần 6% chỉ bằng cách luôn dự đoán No, bất kể giá trị predictor! Đây được gọi là null rate.

null rate

Giả sử có một chi phí không nhỏ khi cố bán bảo hiểm cho một cá nhân. Chẳng hạn, có thể nhân viên bán hàng phải đến gặp từng khách hàng tiềm năng. Nếu công ty cố bán bảo hiểm cho một tập khách hàng được chọn ngẫu nhiên thì tỷ lệ thành công chỉ khoảng 6%, có thể quá thấp so với chi phí bỏ ra. Thay vào đó, công ty muốn chỉ cố bán bảo hiểm cho những khách hàng có khả năng mua. Vì vậy overall error rate không phải điều ta quan tâm. Thay vào đó, điều đáng quan tâm là tỷ lệ những cá nhân được dự đoán đúng là sẽ mua bảo hiểm.

In [58]:

```
confusion_table(knn1_pred, y_test)
```

Out[58]:

	Truth	No	Yes
Predicted			
No		880	58
Yes		53	9

Hóa ra KNN với $K = 1$ tốt hơn random guessing rất nhiều trong nhóm khách hàng được dự đoán sẽ mua bảo hiểm. Trong 62 khách hàng như vậy, có 9 người, tức 14.5%, thực sự mua bảo hiểm. Tỷ lệ này gấp đôi tỷ lệ thu được từ random guessing.

In [59]:

```
9/(53+9)
```

Out[59]:

0.145

Tuning Parameters

Số neighbor trong KNN được gọi là một tuning parameter, cũng gọi là hyperparameter. Ta không biết trước nên dùng giá trị nào. Vì vậy, điều đáng quan tâm là xem classifier hoạt động thế nào trên test data khi ta thay đổi các parameter này. Có thể thực hiện việc đó bằng một for loop, được mô tả trong Mục 2.3.8. Ở đây ta dùng một for loop để xem accuracy của classifier trong nhóm được dự đoán sẽ mua bảo hiểm thay đổi thế nào khi số neighbor chạy từ 1 đến 5:

tuning
parameter
hyper-
parameter

In [60]:

```
for K in range(1,6):
    knn = KNeighborsClassifier(n_neighbors=K)
    knn_pred = knn.fit(X_train, y_train).predict(X_test)
    C = confusion_table(knn_pred, y_test)
    templ = ('K={0:d}: # predicted to rent: {1:>2}, ' +
            ' # who did rent {2:d}, accuracy {3:.1%} ')
    pred = C.loc['Yes'].sum()
    did_rent = C.loc['Yes', 'Yes']
    print(templ.format(
        K,
        pred,
        did_rent,
        did_rent / pred))
```

Out[60]:

```
K=1: # predicted to rent: 62, # who did rent 9, accuracy 14.5%
K=2: # predicted to rent: 6, # who did rent 1, accuracy 16.7%
K=3: # predicted to rent: 20, # who did rent 3, accuracy 15.0%
K=4: # predicted to rent: 3, # who did rent 0, accuracy 0.0%
K=5: # predicted to rent: 7, # who did rent 1, accuracy 14.3%
```

Ta thấy có một mức variability nhất định — các con số với $K = 4$ rất khác phần còn lại.

Comparison to Logistic Regression

Để so sánh, ta cũng có thể fit một logistic regression model cho dữ liệu. Việc này cũng có thể làm bằng **sklearn**, mặc dù theo mặc định nó fit một thứ giống ridge regression version của logistic regression, sẽ được giới thiệu trong Chương 6. Có thể thay đổi điều này bằng cách đặt argument **C** phù hợp như bên dưới. Giá trị mặc định của nó là 1, nhưng khi đặt thành một số rất lớn, algorithm hội tụ về cùng solution như estimator logistic regression thông thường (unregularized) đã thảo luận ở trên.

Khác với package **statsmodels**, **sklearn** tập trung ít hơn vào inference và nhiều hơn vào classification. Do đó, các summary method thấy trong **statsmodels** và version đơn giản hóa của ta dùng **summarize** thường không có cho classifier trong **sklearn**.

In [61]:

```
logit = LogisticRegression(C=1e10, solver='liblinear')
logit.fit(X_train, y_train)
logit_pred = logit.predict_proba(X_test)
logit_labels = np.where(logit_pred[:,1] > 5, 'Yes', 'No')
confusion_table(logit_labels, y_test)
```

Out[61]:

	Truth	No	Yes
Predicted			
No		933	67
Yes		0	0

Ta dùng argument **solver='liblinear'** ở trên để tránh warning với default solver, warning này sẽ cho biết algorithm không converge.

Nếu dùng 0.5 làm predicted probability cut-off cho classifier thì ta gặp một vấn đề: không test observation nào được dự đoán sẽ mua insurance. Tuy nhiên, ta không bắt buộc phải dùng cut-off 0.5. Nếu thay vào đó ta dự đoán một khách hàng sẽ mua bất cứ khi nào predicted probability của việc mua vượt 0.25, ta thu được kết quả tốt hơn nhiều: ta dự đoán 29 người sẽ mua bảo hiểm và đúng với khoảng 31% trong số đó. Con số này tốt hơn random guessing gần năm lần!

In [62]:

```
logit_labels = np.where(logit_pred[:,1]>0.25, 'Yes', 'No')
confusion_table(logit_labels, y_test)
```

Out[62]:

	Truth	No	Yes
Predicted			
No		913	58
Yes		20	9

In [63]:

```
9/(20+9)
```

Out[63]:

```
0.310
```